

Quick-Start: AI & RAG Guide

This guide will walk you through setting up your local environment, verifying your API access, and building a PDF-based AI Chatbot.

1. Install the Toolkit

With your environment active, run this single command to install everything:

```
pip install langchain-openai langchain-community langchain-core pdfminer.six
chromadb streamlit tiktoken httpx
```

2. Test Your API Key

Let's make sure your "Passport" (API Key) works. Create a file named `test_key.py` and paste this code:

```
import httpx
from langchain_openai import ChatOpenAI

# Lab Setup: Ignore SSL certificate errors
client = httpx.Client(verify=False)

llm = ChatOpenAI(
    base_url="https://genailab.tcs.in",
    model="azure/genailab-maas-gpt-4.1",
    api_key="YOUR_PROVIDED_KEY_HERE",
    http_client=client
)

try:
    response = llm.invoke("Hi! Are you online?")
    print("✅ CONNECTION SUCCESSFUL!")
    print("AI Response:", response.content)
except Exception as e:
    print("❌ ERROR: Could not connect. Check your key or internet.")
    print(f"Details: {e}")
```

To run it: `python test_key.py`

3. Build the RAG Chatbot

RAG (Retrieval-Augmented Generation) allows the AI to look at your specific data (like a PDF) to answer questions.

Create a file named `app.py` and paste the following:

```

import hashlib
import os
import tempfile

# --- Runtime environment fixes ---
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
os.environ["STREAMLIT_SERVER_WATCH_VARIABLE_NAMES"] = "false"
os.environ["CURL_CA_BUNDLE"] = ""
os.environ["PYTHONHTTPSVERIFY"] = "0"

import httpx
import requests
import streamlit as st
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from pdfminer.high_level import extract_text
from urllib3.exceptions import InsecureRequestWarning

# Silence SSL warnings in the console
requests.packages.urllib3.disable_warnings(category=InsecureRequestWarning)

API_KEY = "sk-gzobU7EMMkrAECyIhQyb-A"
BASE_URL = "https://genailab.tcs.in"

PROMPT_TEMPLATE = """Answer the question based only on the context provided.
Context: {context}
Question: {question}
Answer: """

@st.cache_resource
def get_llm_and_embeddings():
    """Create model clients once and reuse them across interactions."""
    client = httpx.Client(verify=False)

    llm = ChatOpenAI(
        base_url=BASE_URL,
        model="azure/genailab-maas-gpt-4.1",

```

```

        api_key=API_KEY,
        http_client=client,
        temperature=0,
    )

    embeddings = OpenAIEmbeddings(
        base_url=BASE_URL,
        model="azure/genailab-maas-text-embedding-3-large",
        api_key=API_KEY,
        http_client=client,
        check_embedding_ctx_length=False,
    )
    return llm, embeddings

def extract_text_from_uploaded_pdf(file_bytes: bytes) -> str:
    """Extract text from PDF bytes using a temp file path."""
    temp_path = None
    try:
        with tempfile.NamedTemporaryFile(delete=False, suffix=".pdf") as tmp:
            tmp.write(file_bytes)
            temp_path = tmp.name
        return extract_text(temp_path)
    finally:
        if temp_path and os.path.exists(temp_path):
            os.remove(temp_path)

def build_retriever(text: str, embeddings):
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=1000,
        chunk_overlap=100,
        length_function=len,
    )
    chunks = splitter.split_text(text)
    vectorstore = FAISS.from_texts(chunks, embeddings)
    return vectorstore.as_retriever(search_kwargs={"k": 3})

def build_rag_chain(retriever, llm):
    prompt = ChatPromptTemplate.from_template(PROMPT_TEMPLATE)
    return (
        {

```

```

        "context": retriever | (lambda docs: "\n\n".join(d.page_content
for d in docs)),
        "question": RunnablePassthrough(),
    }
    | prompt
    | llm
    | StrOutputParser()
)

def clear_rag_session():
    for key in ("retriever", "rag_chain", "file_id"):
        st.session_state.pop(key, None)

def main():
    st.set_page_config(page_title="Hardened RAG Bot", layout="wide")
    st.title("📄 PDF AI Explorer")
    st.caption("Running in Corporate-Safe Mode (No Tiktoken, No SSL Checks)")

    uploaded_file = st.file_uploader("Upload your PDF document", type="pdf")
    if not uploaded_file:
        st.info("Please upload a PDF to get started.")
        clear_rag_session()
        return

    file_bytes = uploaded_file.getvalue()
    file_id = f"{uploaded_file.name}:{len(file_bytes)}:
{hashlib.md5(file_bytes).hexdigest()}"

    if st.session_state.get("file_id") != file_id:
        with st.spinner("Processing PDF (Extracting & Indexing)..."):
            try:
                text = extract_text_from_uploaded_pdf(file_bytes)
                if not text.strip():
                    st.error("No readable text found in the PDF.")
                    clear_rag_session()
                    return

                llm, embeddings = get_llm_and_embeddings()
                retriever = build_retriever(text, embeddings)
                rag_chain = build_rag_chain(retriever, llm)

```

```

        st.session_state.file_id = file_id
        st.session_state.retriever = retriever
        st.session_state.rag_chain = rag_chain
        st.success("Document successfully indexed!")
    except Exception as e:
        clear_rag_session()
        st.error(f"Error processing PDF: {e}")
        return

if "rag_chain" in st.session_state:
    st.divider()
    query = st.text_input("Enter your question about the document:")

    if query:
        with st.spinner("Generating answer..."):
            try:
                response = st.session_state.rag_chain.invoke(query)
                st.markdown("### Answer")
                st.info(response)
            except Exception as e:
                st.error(f"Error during query: {e}")

if __name__ == "__main__":
    main()

```

4. Launch Your Application

To see your chatbot in the browser, run this command in your terminal:

```
streamlit run app.py
```